

Materia: Programación Orientada a Objetos	Código: 72.33
Créditos: 6	Carga horaria total: 102
Departamento: Sistemas Digitales y Datos	Año: 2023
Carrera de: Ingeniería en Informática / Ingeniería Mecánica	
Fecha última actualización: 21/12/2022 12:45:40	

➤ Carga horaria:

102	Horas totales
51	hs. teóricas
34	hs. prácticas
17	hs. de laboratorio

6	Horas semanales
6	hs. presencial
0	hs. a distancia

➤ Contenidos mínimos:

Tipo Abstracto de Datos: ocultamiento de la información y encapsulamiento. Características del Paradigma Objetos: encapsulamiento, ocultamiento, herencia y polimorfismo. Mensajes y métodos. Clases, subclases e instancias. Representación interna de objetos y clases. Chequeo de tipos. Binding estático y dinámico. Uso y construcción de bibliotecas. Colecciones (arreglos) e iteradores. Manejo de excepciones. Persistencia de objetos. Introducción a UML para el diseño de clases e interfaces. Comparación de paradigmas de programación. Taller de aplicación con un lenguaje orientado a objeto tipo Smalltalk o Java.

➤ Presentación de la materia:

Esta materia del departamento de Sistemas Digitales y Datos se dicta en el segundo año de la carrera de Ingeniería Informática del ITBA. Es un curso de programación enfocado en el paradigma de la programación orientada a objetos aplicado a dos lenguajes: Java y Ruby.

El propósito central es resaltar las ventajas del paradigma orientado a objetos respecto al paradigma imperativo que los alumnos ya conocen. Los conocimientos del paradigma y del lenguaje Java son fundamentales para materias posteriores de la carrera donde también se abordan conceptos de programación.

La materia está pensada para alumnos con conocimientos del paradigma de la programación imperativa y los mismos son tomados como base para los conceptos del paradigma orientado a objetos de forma que el alumno pueda resolver problemas del mundo real utilizando jerarquías de objetos.

➤ Objetivos de aprendizaje:

- Distinguir las ventajas de la utilización del paradigma de la programación orientada a objetos frente a otros paradigmas.
- Diseñar e implementar jerarquías de objetos que sean una abstracción del mundo real y permitan resolver problemas.
- Generar código de calidad, haciendo foco en la reutilización del mismo, siguiendo las guías estilo recomendadas.
- Incorporar los aspectos principales de Java para aplicarlo como lenguaje de programación orientado a objetos, a partir de la utilización de clases, interfaces y enumerativos.
- Incorporar los aspectos principales de Ruby para aplicarlo como lenguaje de programación orientado a objetos, a partir de la utilización de clases y módulos.

➤ Contenidos:

Título	Descripción
Introducción a la programación orientada a objetos (OOP: Object Oriented Programming)	Antecedentes: Tipo abstracto de datos (TAD) Ocultamiento de la información, encapsulamiento, polimorfismo y herencia Terminología: Clases (moldes), objetos (instancias), métodos (mensajes entre objetos). Variables de instancia y de clase. Métodos de instancia y de clase (binding dinámico y estático). Clases abstractas. Clases inmutables. Composición y herencia. Subclases y superclases. Sobrecarga y redefinición de métodos. Tipos de polimorfismo. Método constructor. Conceptos básicos de UML (Unified Modeling Language).
Introducción a Java	Características generales de Java como lenguaje orientado a objetos. Bytecode. Máquina Virtual. Package. Garbage Collection. Tipado estático. Modificadores de visibilidad de las variables y los métodos. Clase Object. Operador instanceof. Igualdad y equivalencia (método equals). Método toString. Autoboxing y unboxing. Control de flujo. Loops. Arreglos. String y StringBuilder. Manejo de errores. Excepciones verificadas y no verificadas. Requerimiento catch or specify.
Enumerativos en Java	Enums con variables de instancia y métodos abstractos. Métodos comunes a todos los enums.
Interfaces y tipos de datos genéricos en Java	Comparación entre interfaz y clase abstracta. Métodos default en interfaces. Interfaces funcionales. Expresiones lambda. Tipo genérico (unbounded, upper bounded y lower bounded). Orden natural y orden particular (interfaces Comparable y Comparator). Wildcards.
Clases anidadas en Java	Inner class, static nested class y clases anónimas.
Iteradores en Java	Interfaces Iterable e Iterator.

Anotaciones en Java	Diseño de Annotations. Target y retention.
Pruebas de unidad en Java	Unit Testing. Test Coverage. JUnit 5.
Colecciones en Java	Java Collections Framework: Interfaces Collection, List, Set, Map, Queue, Deque y sus implementaciones. Hashing y resolución de colisiones.
Introducción a Ruby	Características generales de Ruby como lenguaje orientado a objetos. Tipado dinámico. Modificadores de visibilidad de los métodos. Igualdad y equivalencia. Método to_s. Control de flujo. String. Rangos. Bloques. Loops. Manejo de errores. Excepciones. Módulos.
Colecciones en Ruby	Clases Array, Set y Hash. Módulos Enumerable y Comparable. Iteradores.
Interfaz gráfica en Java	Introducción a JavaFX. Concepto de eventos y handlers.
Streams en Java	Introducción al paradigma funcional. Method References. Interfaces funcionales de la librería de Java. Interfaz Stream. Clase Optional. Manejo de archivos. Persistencia de objetos. Alternativas a la serialización por defecto.

➤ Estrategias de enseñanza:

- Resolución de problemas: aplicada a los ejercicios prácticos cubiertos durante las clases, los trabajos prácticos y los exámenes parciales
- Aprendizaje basado en proyectos: aplicado en el proyecto final de la materia

➤ Actividades:

Título	Descripción
Ejercicios prácticos	Durante las clases se desarrollan ejercicios para que los alumnos planteen de forma individual o grupal la resolución del problema. Luego se realiza una puesta en común analizando las ventajas y desventajas de las soluciones propuestas por los alumnos.
Trabajos prácticos	Los alumnos aplican los conocimientos adquiridos a través de la resolución de los ejercicios presentes en las guías de trabajos prácticos. Dichos ejercicios están especialmente pensados para ayudar al aprendizaje de los puntos importantes en la materia. La modalidad de trabajo es individual y se espera que el alumno aplique los siguientes pasos en la resolución de ejercicios: - Análisis del problema - Planteo de solución - Implementación y prueba del programa Los trabajos prácticos se describen a continuación: TP 01: Introducción a la POO TP 02: Introducción a Java TP 03: Introducción a la POO en Java TP 04: Interfaces y Excepciones en Java TP 05: Tipos Genéricos en Java TP 06: Colecciones en Java TP 07: Diseño Avanzado en Java TP 08: Introducción a Ruby TP 09: Colecciones en Ruby TP 10: Diseño Avanzado en Ruby
Lecturas recomendadas	Se presentan enlaces web con material sugerido para que los alumnos consulten antes de las clases

➤ Recursos didácticos:

- Presentaciones:
 - Material de las clases teórico/prácticas
 - Material de las clases de taller/laboratorio
- Trabajos Prácticos
- Lecturas recomendadas

- Software utilizado:
 - JDK: Java Development Kit
 - SDK Ruby
 - JetBrains IntelliJ IDEA
 - JetBrains RubyMine

Todos los recursos se publican en el Campus ITBA.

➤ Modalidad de evaluación y aprobación:

Modalidad de evaluación:

La evaluación está centrada en dos exámenes parciales (P1, P2) individuales con desarrollo escrito. Los parciales se aprueban con una nota de 5 o más puntos (en una escala de 1 al 10).

Los exámenes consisten en ejercicios de implementación donde se cuenta con un programa de prueba y una salida esperada. Los estudiantes deben implementar todo lo necesario para que, de ejecutarse el programa utilizando la implementación del alumno, se obtenga la salida esperada. Se puede solicitar también a los alumnos que completen partes del programa de prueba para que se logre la salida indicada.

Los alumnos deben además completar durante la cursada las autoevaluaciones individuales con consignas de opción múltiple con el propósito de realizar un diagnóstico de los contenidos aprendidos en cada unidad. Las mismas pueden ser realizadas en múltiples intentos, quedando como calificación definitiva la del intento con la calificación más alta antes de la fecha de vencimiento.

Al final de la cursada los alumnos son evaluados con la realización de un proyecto grupal de implementación. Este proyecto consiste en la implementación de una aplicación Java con interfaz gráfica donde el grupo ya cuenta con una implementación inicial provista por la cátedra. El objetivo es incorporarle mayor funcionalidad siempre siguiendo los principios del paradigma, sobre todo aprovechando la reutilización del código y respetando una correcta separación entre el front-end y el back-end.

Requisitos de aprobación:

Son requisitos de aprobación de la cursada:

- Aprobar los dos exámenes parciales (P1, P2), o sus recuperatorios.
- Haber completado todas las autoevaluaciones antes de la fecha de vencimiento (sin calificación mínima)

¿Cómo se calcula la nota de cursada?

- Nota de cursada = $0.9 * \text{Nota de Parciales} + 0.1 * \text{Notas de autoevaluaciones}$
- En caso de aprobar el recuperatorio de un parcial la nota del mismo se obtiene como: $0.25 * \text{Nota del parcial} + 0.75 * \text{Nota del recuperatorio}$

➤ Bibliografía obligatoria:

N°	Descripción
1	Oracle, "Learn Java", dev.java. https://dev.java/learn/ (accessed Sep. 18, 2022).
2	Techotopia, "Ruby Essentials", techotopia.com https://www.techotopia.com/index.php/Ruby_Essentials (accessed Sep. 18, 2022).

➤ **Bibliografía complementaria:**

N°	Descripción
1	Bloch, Joshua. Effective Java. Third edition, Addison-Wesley, 2018.
2	Metz, Sandi. Practical object-oriented design in Ruby: an agile primer. Addison-Wesley, 2013.